

35 | 秒级开发体验，如何实现容器热加载和一键调试？

王炜 · 云原生架构与GitOps实战



你好，我是王炜。

在上一节课，我为你介绍了 GitOps 开发循环反馈变慢的原因以及三种解决方案。在这三种解决方案中，我推荐你使用远程开发的方式来加速开发循环反馈。

这节课，我们就来学习如何借助 Nocalhost 实现 Kubernetes 应用秒级的开发体验，提升开发循环反馈效率。

Nocalhost 是腾讯云 CODING 在 2020 年开源的项目，同时它也是云原生开发领域下第一个由国人主导并进入 CNCF Sandbox 的项目。

我有幸参与了 Nocalhost 从 0 到 1 的研发过程，也深知其工作原理。所以，这节课除了带你了解 Nocalhost 的基本使用以外，我还会带你深入了解它的工作模式和原理，让你更深刻地理解“远程开发”这种创新的开发模式。

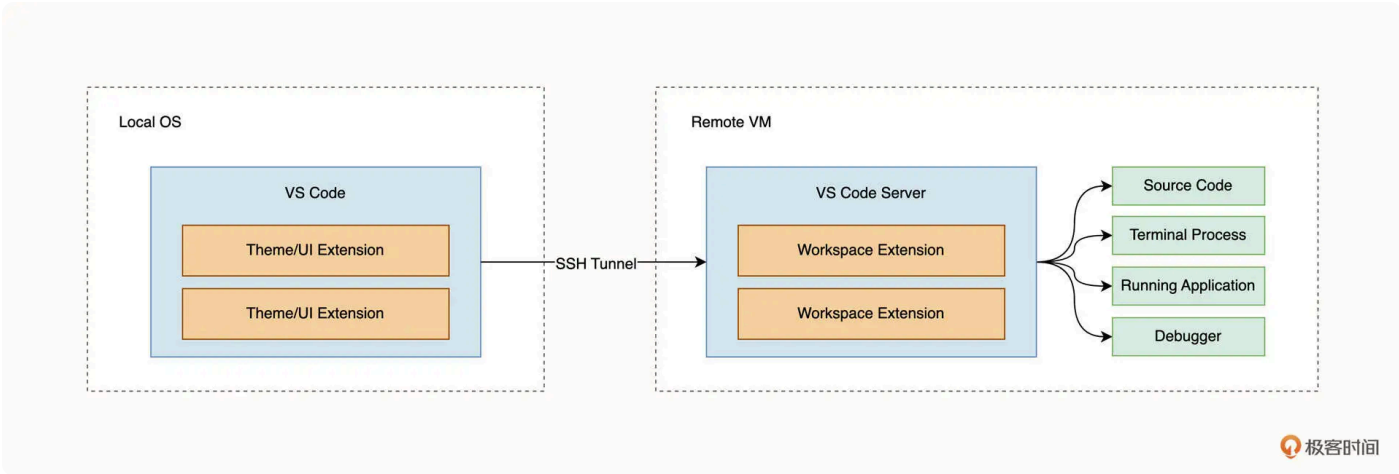
在开始今天的学习之前，你需要做好下面这些准备。

按照第一章 [第 2 讲](#) 的内容在本地通过配置文件创建 Kind 集群，并在集群内安装好 Ingress-nginx。

配置 Kubectl 使其能够访问 Kind 集群。

远程开发概述

说起远程开发，我相信有一些同学会想到 VS Code 的 SSH Remote。它利用了 VS Code 客户端和 Server 解耦的特性，让 VS Code Server 运行在远端的 VM，本地 VS Code 客户端只起到了 UI 展示和交互的作用，两者通过 SSH Tunnel 进行数据交互，它的工作原理如下图所示。



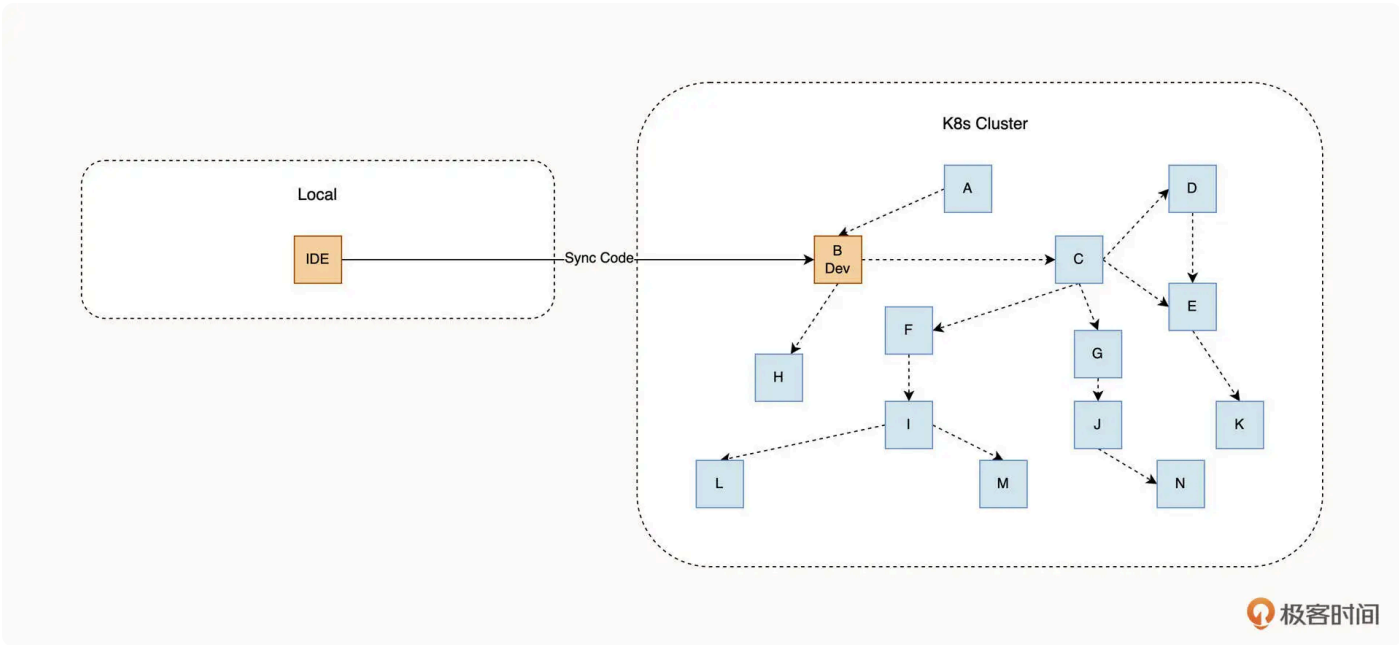
这种开发方式主要有两个好处。首先，源代码并不会在本地保存，我们在本地编辑的代码实际上是远端机器上的代码，这对于源码安全性要求较高的团队来说是一种不错的实践。其次，由于应用的进程是运行在远端机器上的，所以我们可以借助云端的资源来编译和运行应用，这解决了大型单体应用在本地产开时的资源限制问题。

这种远程开发的思路具有创新性，那么，在云原生架构下我们能否参考这种思想呢？

在学习容器技术的时候，我们经常会把容器比作一台 VM 虚拟机，如果将虚拟机替换成 Pod，似乎就可以实现远程开发的效果了。

但事实真的是这样吗？我们来继续分析。在 Kubernetes 架构体系下，由于 Pod 并没有外网 IP，所以我们很难通过 SSH 的方式连接 Pod。其次，在镜像里内置 SSH Server 并开启 SSH 访问也容易造成安全隐患，我们需要用其他的连接方式取代 SSH。

我们来回顾一下上一节课中我提到的在容器内远程开发的方式，也就是 Nocalhost 的开发方式，如下图所示。



如果你仔细分析这两种开发方式的差异，你会发现，Nocalhost 用代码同步取代了 SSH，这样也就解决了我们上面说的 SSH 连接的问题。

Nocalhost 开发实战

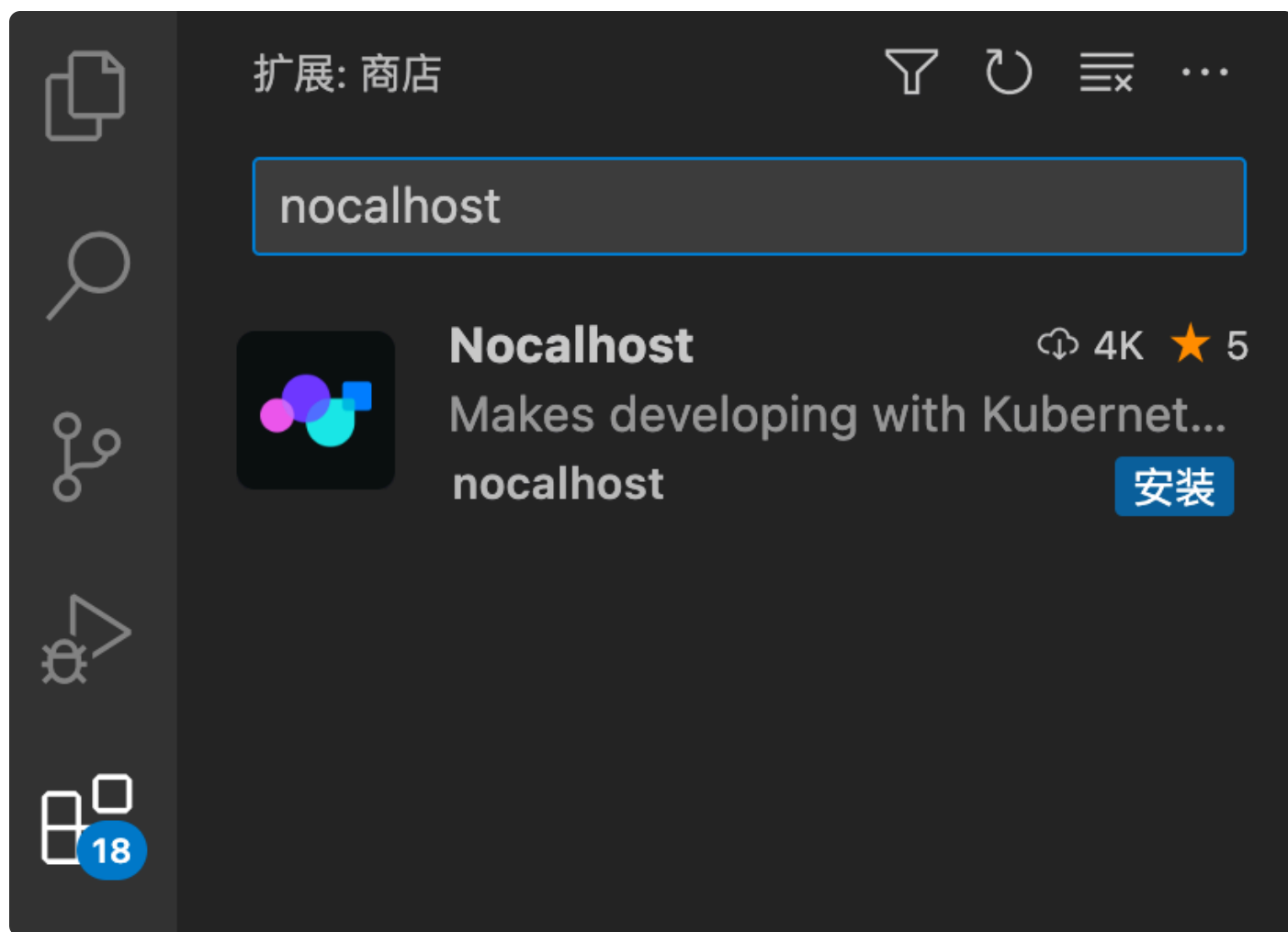
到这里，远程开发的基本知识就介绍完了。接下来我们进入实战环节。

在进入实战之前，你需要先安装 Nocalhost IDE 插件。Nocalhost 支持 VS Code 和 JetBrains 全系列的 IDE，你可以在市场中搜索。

安装 Nocalhost

接下来，我以 VS Code 为例简单介绍如何安装 Nocalhost 插件。Jetbrains 插件的安装你可以参考 [这个文档](#)。

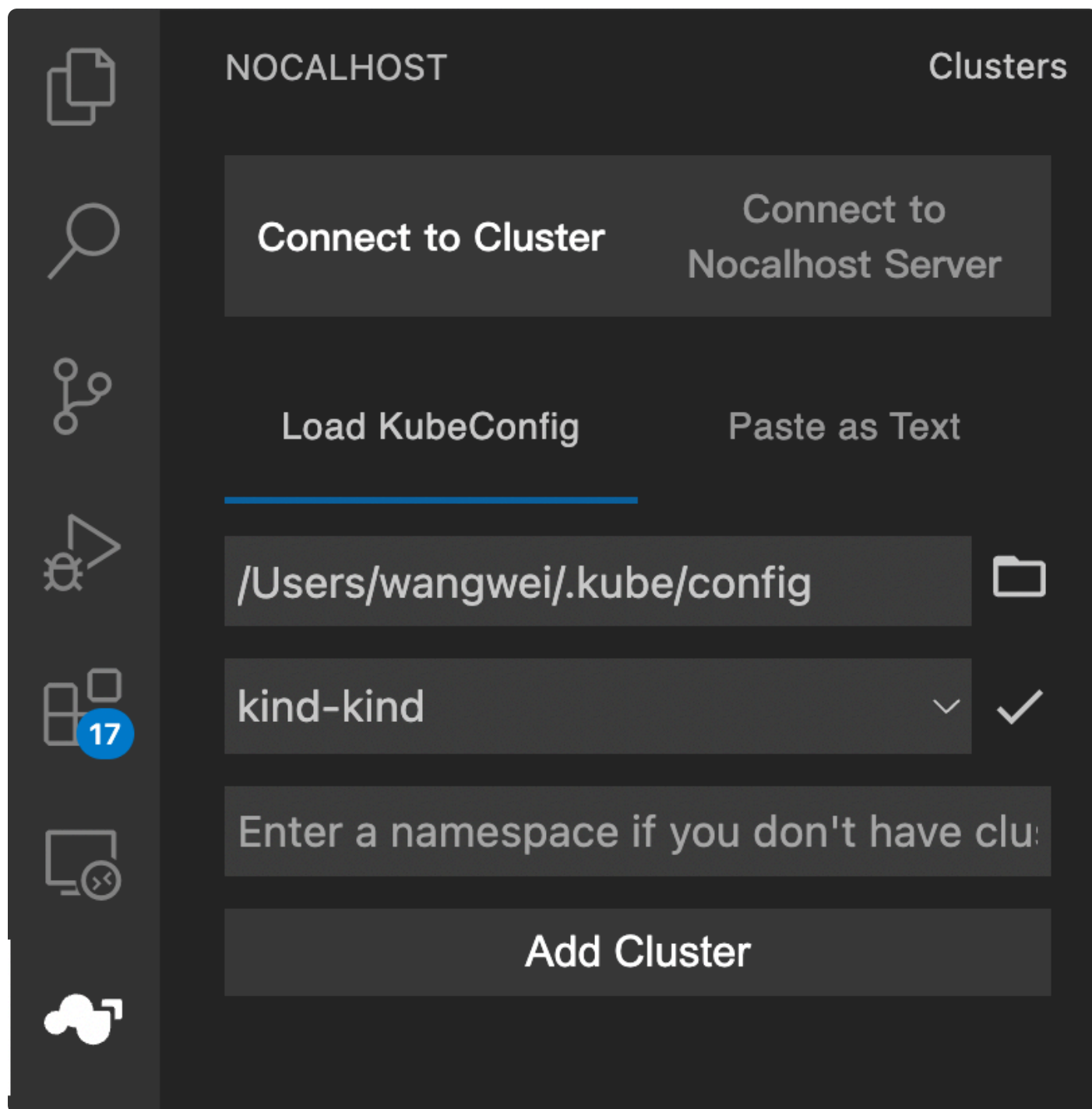
首先，在 VS Code 插件市场中搜索 Nocalhost，然后点击“安装”按钮进行安装，如下图所示。



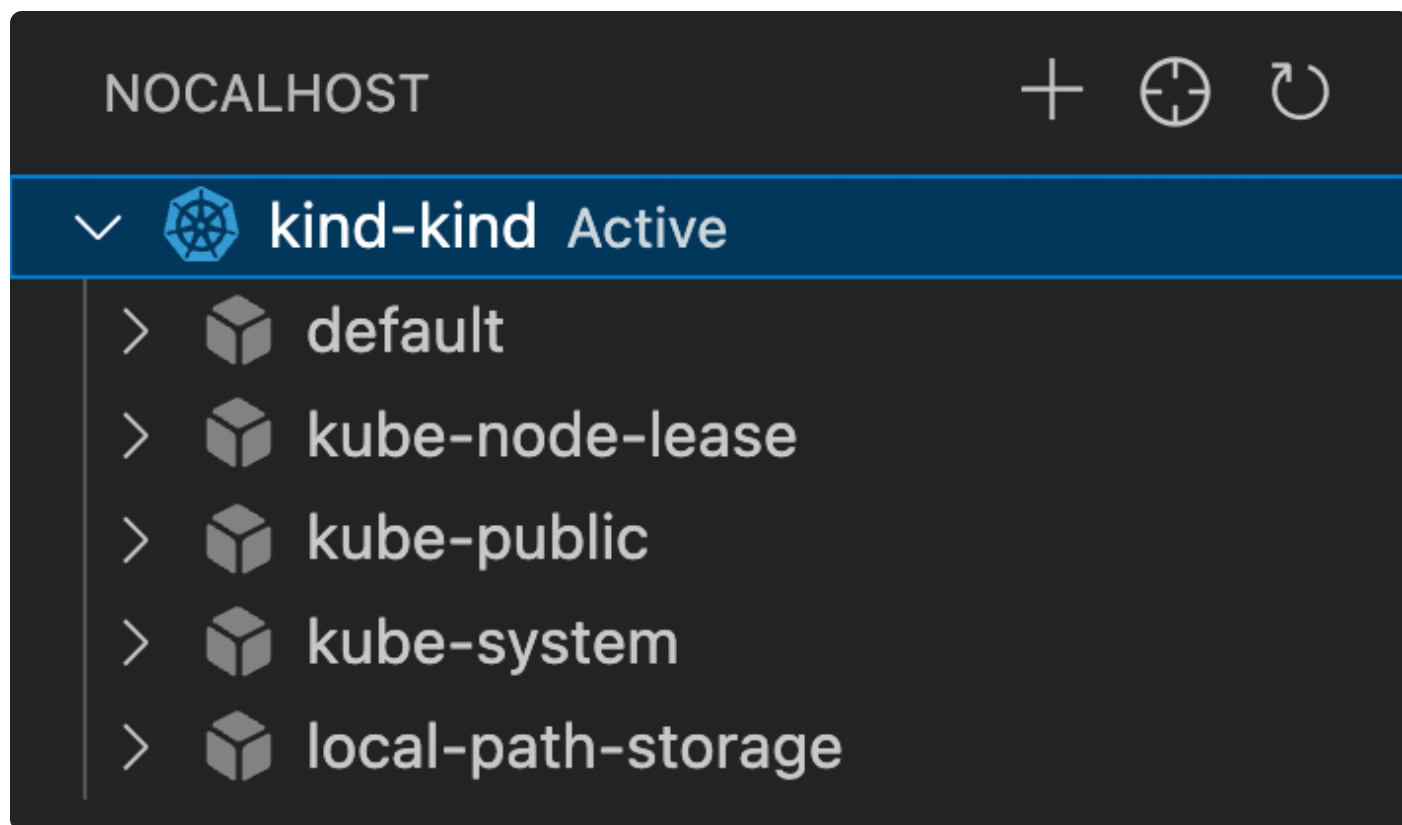
在安装 IDE 插件之后，Nocalhost 会自动下载 nhctl 工具，你可以在 VS Code 的右下角查看下载进度，nhctl 是 Nocalhost 的核心组件，它为插件提供 Kubernetes API 调用能力。

添加 Kubernetes 集群

接下来，在 VS Code 左侧菜单栏中打开 Nocalhost 插件，如果你已经提前准备好了 Kind 集群，Nocalhost 就会自动识别，点击“Add Cluster”即可添加集群。



添加完成后，你可以点击集群名称来查看集群的命名空间，这里将以树状结构展示命名空间，如下图所示。



部署示例应用

添加完集群后，就可以部署 Nocalhost 提供的示例应用了。

将鼠标移动到“default”命名空间，点击右侧的“火箭”按钮，在弹出的对话框中，选择“Deploy Demo”选项来部署示例应用。

**Please select the installation
source of application.**

Deploy From Local Directory

Deploy From Git Repo

Deploy From Helm Repo

Deploy Demo

此时，Nocalhost 将自动从 GitHub 克隆示例应用仓库，并将它部署到集群的 default 命名空间下。同时，在 VS Code 输出栏中会出现等待 Pod 就绪的提示信息。

当弹出下面的提示时，说明示例应用已经就绪了。

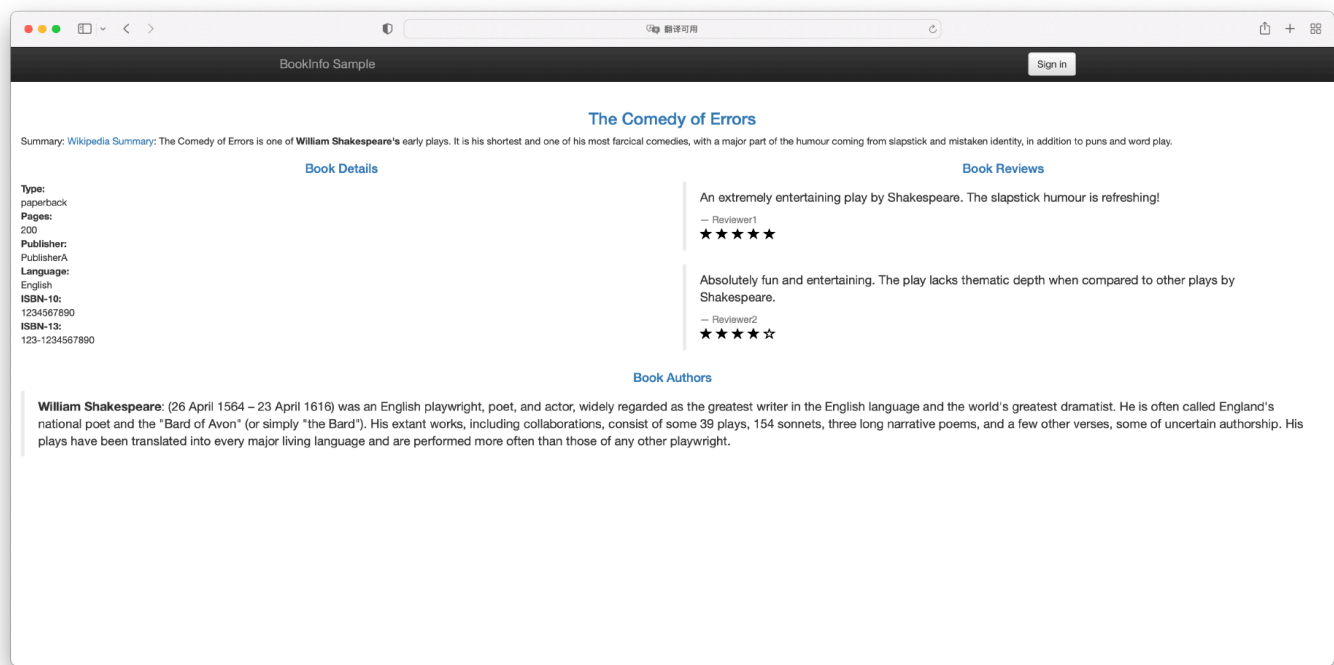


**Do you want to open the browser
to access application?**

取消

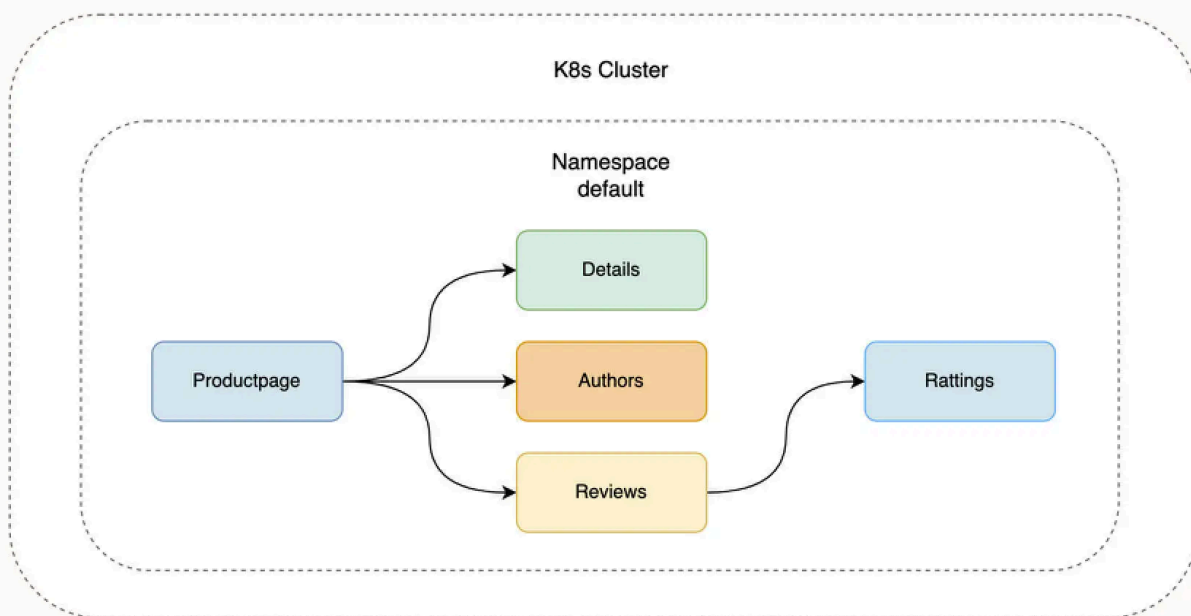
go

这时候，点击“go”按钮来打开示例应用，Nocalhost 将自动进行端口转发，并打开浏览器访问 <http://127.0.0.1:39080/productpage> 示例应用页面，如下图所示。



这里我简单介绍一下这个示例应用，这是一个图书管理系统，展示了书籍的详情信息、评价、作者信息、评分。

每部分信息都是由不同的微服务输出的，示例应用一共有 5 个微服务组成，它们分别是 Productpage 服务、Reviews 服务、Details 服务、Ratings 服务和 Authors 服务。其中，Productpage 服务负责输出首页以及请求其他的微服务，也是应用的入口，其他服务根据字面意思分别输出了其他的内容。为了方便你理解它们之间的请求关系，我画了一张图放在了文稿中。

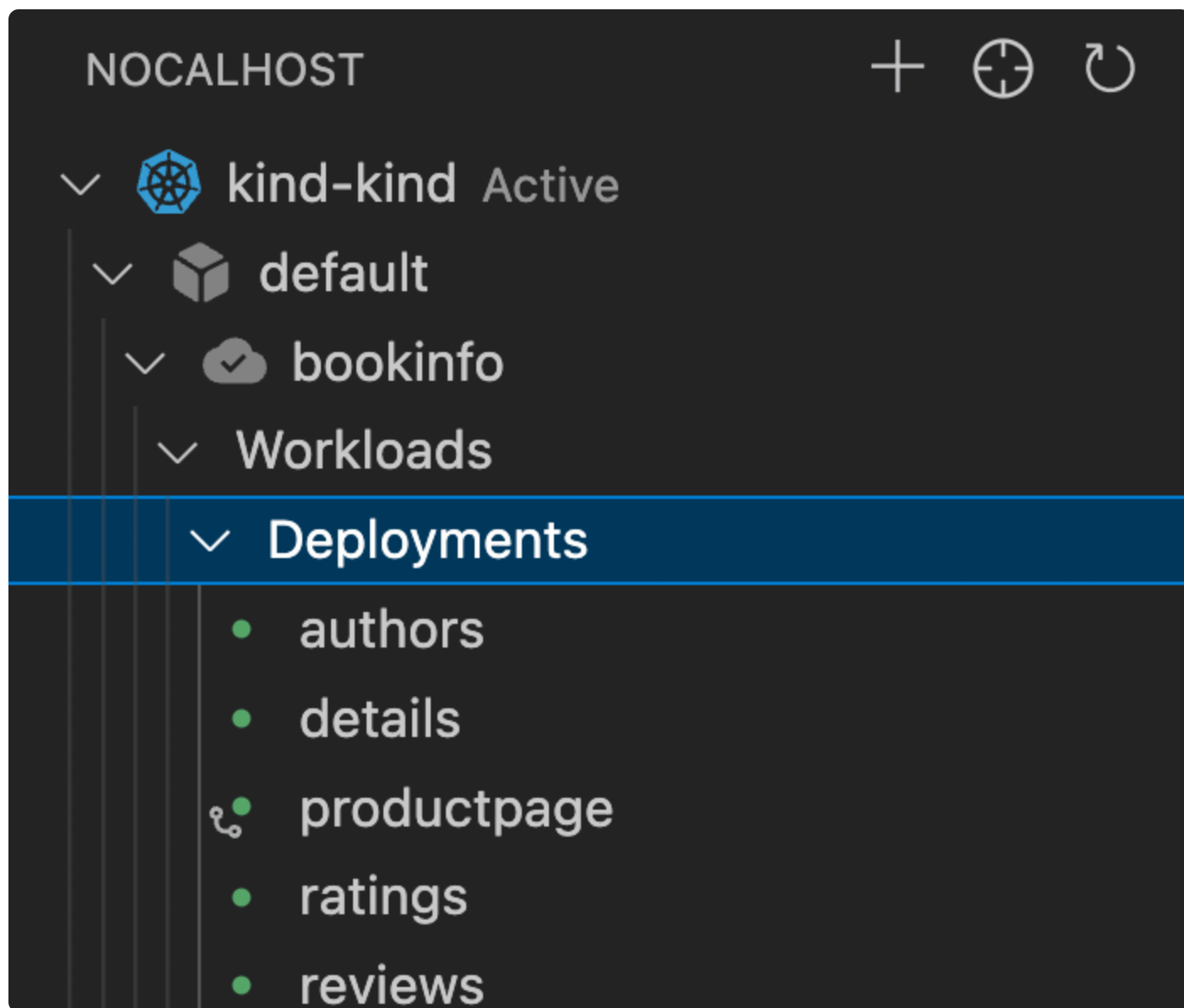


秒级开发循环反馈

在完成示例应用的部署后，假设我们现在有一个需求，我希望修改 Authors 服务输出的作者名信息。一般而言，我需要在本地找到对应的代码并修改，然后构建镜像，推送到镜像仓库，接着修改 Kubernetes 集群的 Authors 工作负载的镜像版本，然后等待新的镜像启动。

接下来我们来看一下如何使用 Nocalhost 打破传统的开发循环反馈，并获得秒级的 Kubernetes 应用开发体验。

我们在 Nocalhost 插件中点击 default 展开命名空间，然后点击 bookinfo 展开应用，点击 Workload 展开工作负载，最后，点击 Deployment 查看工作负载列表。



此时，将鼠标移动到 authors 服务，点击右侧的“绿色锤子”按钮进入该服务的开发模式。然后，在弹出的对话框中选择“Clone from Git Repo”，并选择一个本地目录用来存储源码。点击确认后，Nocalhost 将自动克隆 authors 服务的源码到所选择的目录下，并将源码通过新的 VS Code 窗口打开。

此时，在新的 VS Code 窗口的右下角你会看到 Nocalhost 进入开发模式的提示，等待片刻后，将获得一个**远端容器**的终端。

[问题](#)[输出](#)[调试控制台](#)[终端](#)

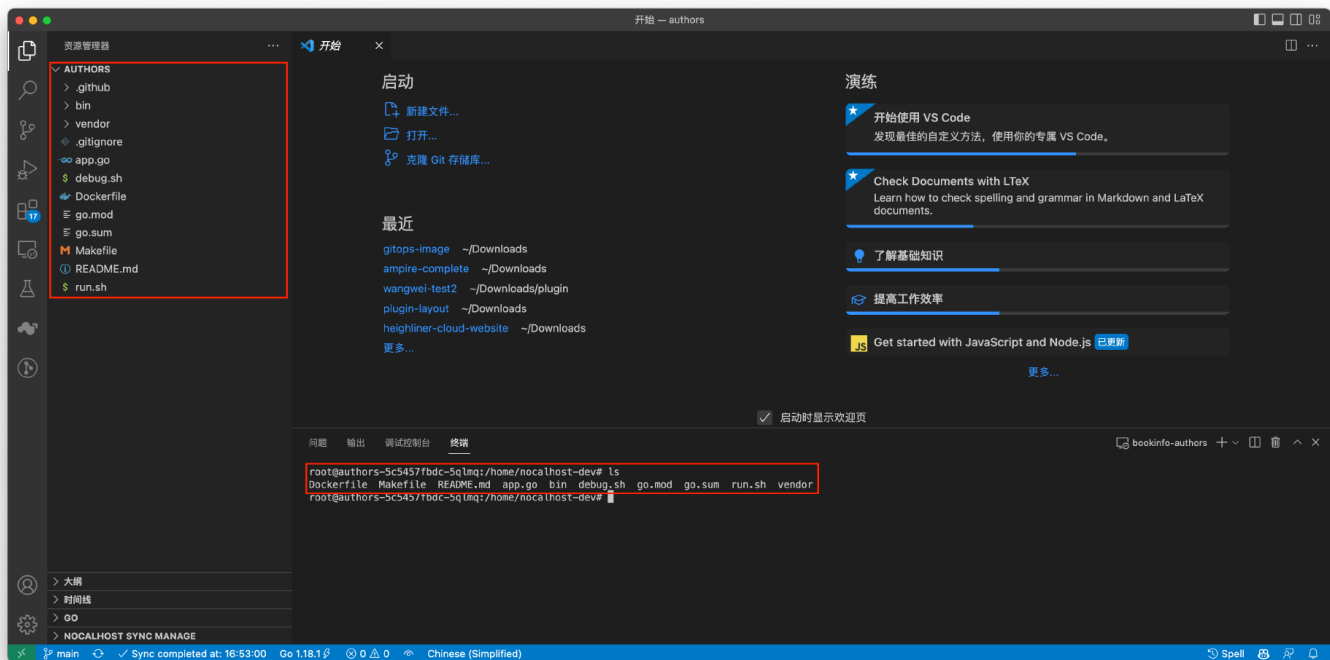
```
root@authors-5c5457fbdc-5qlmq:/home/nocalhost-dev#
```

注意，这个终端并不是本地的终端，而是 authors 服务在开发模式下的终端。也就是说，在此终端下执行的所有命令实际上都是在 authors 服务的容器里执行的。此时，你可以在终端内执行 ls 命令来查看容器的文件目录。

[复制代码](#)

```
1 root@authors-5c5457fbdc-5qlmq:/home/nocalhost-dev# ls
2 Dockerfile  Makefile  README.md  app.go  bin  debug.sh  go.mod  go.sum  run.sh  v
```

通过观察我们会发现，容器内的文件目录和 VS Code 正在编辑的文件目录是一致的。



实际上，容器内的文件和目录就是从 VS Code 打开的源码同步过去的。

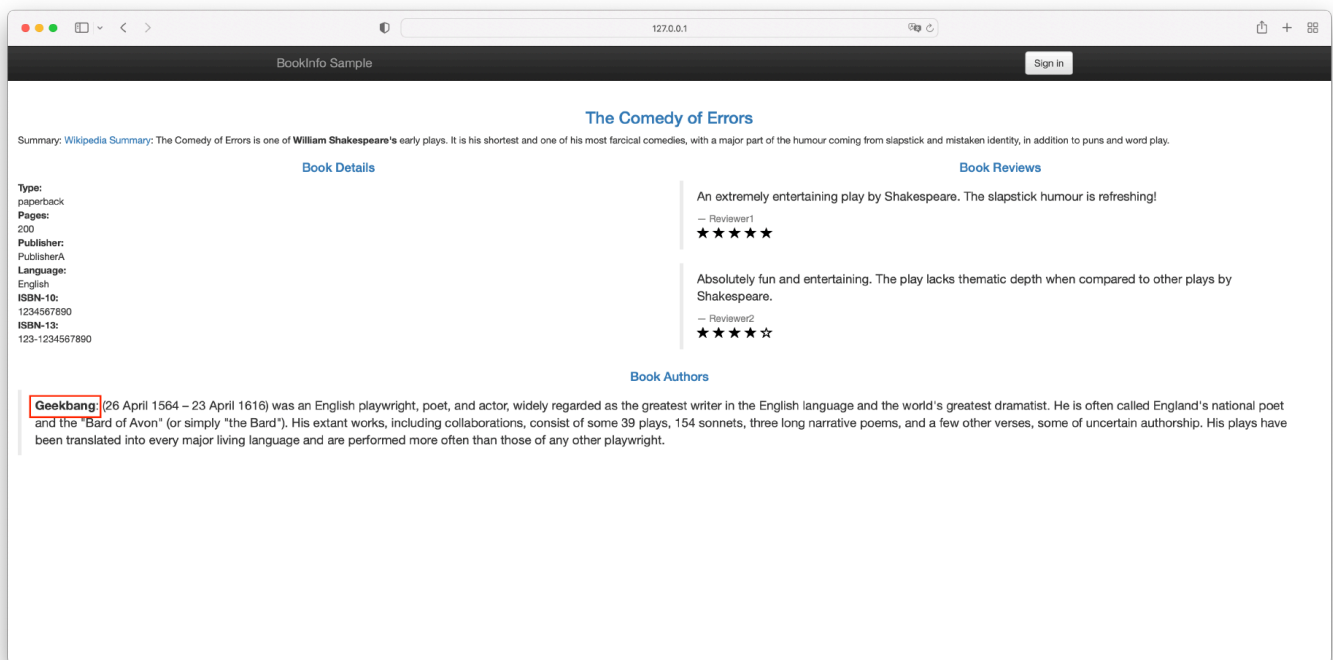
接下来，我们继续最开始提到的修改作者名称的需求。打开 app.go 文件，找到第 53 行，我们尝试将作者信息修改为“Geekbang”，并保存修改。接下来，在终端下运行下面的命令。

 复制代码

```
1 root@authors-5c5457fbdC-5qlmq:/home/nocalhost-dev#sh run.sh
2 2023/01/30 16:58:49 Start listening http port 9080 ...
```

run.sh 脚本实际是 authors 服务的启动命令，它通过 go run app.go 启动了 authors 服务。

当服务启动完成后，接下来，重新返回浏览器并**刷新页面**，你将看到刚才的修改已经实时生效了。



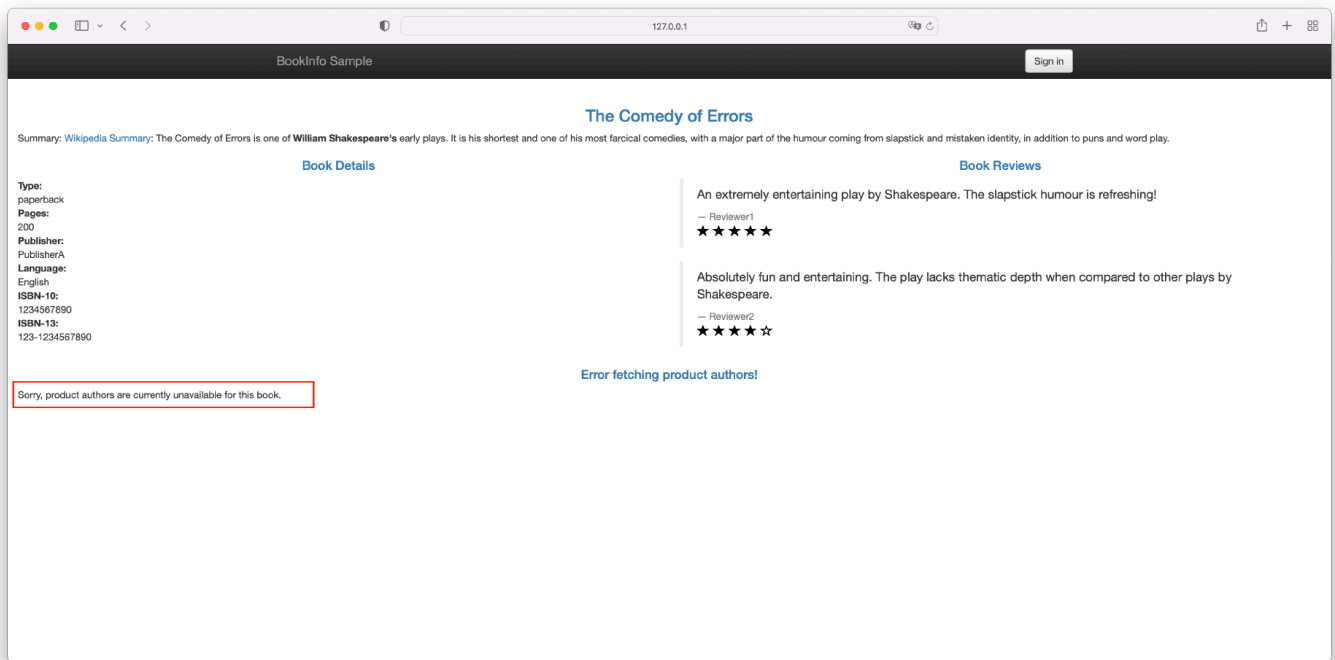
怎么样，是不是觉得开发循环反馈又回到了单体应用时代了呢？

现在，你可以在终端下通过 Ctrl+C 来中断 authors 服务的进程。

```
root@authors-5c5457fbdc-5qlmq:/home/nocalhost-dev# sh run.sh
2023/01/30 16:58:49 Start listening http port 9080 ...
^Csignal: interrupt

root@authors-5c5457fbdc-5qlmq:/home/nocalhost-dev#
```

然后，重新回到浏览器并刷新页面，你将看到 authors 服务当前不可用。



这意味着，我们在进入 authors 服务的开发模式并得到了远程终端之后，便能够直接在容器内控制业务进程的启停操作了。修改代码后，再次重启业务进程就可以得到编码效果，整个体验就像在本地开发一样，它颠覆了传统的开发过程需要构建镜像的操作，并实现了编码实时生效的效果。

容器热加载

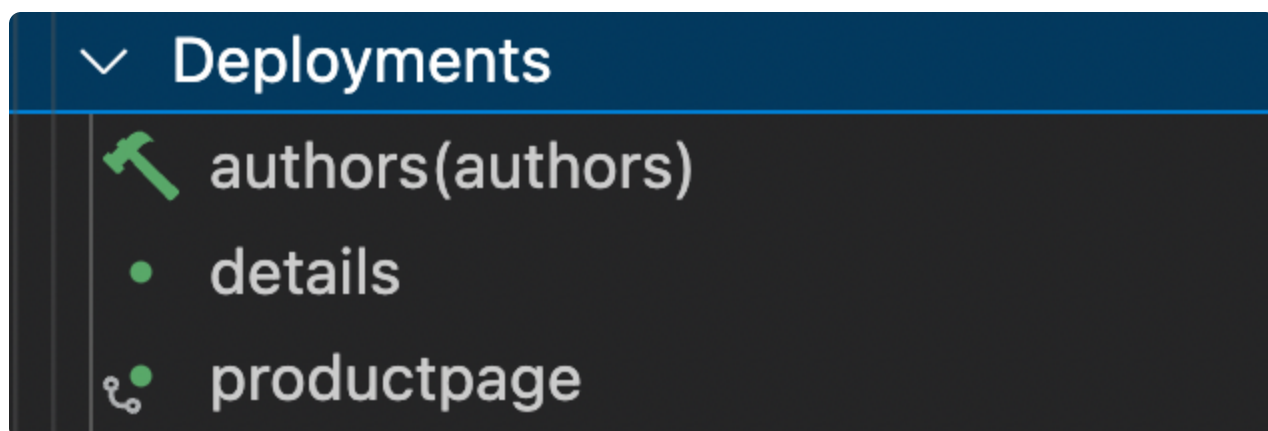
通过上面的介绍，我们知道 Nocalhost 是通过文件同步的技术来实现本地和远端代码一致的，在实际编码过程中，每次在本地修改源码后，我们往往需要手动重启容器内的业务进程才能看到编码效果。

那么，能不能更进一步，实现修改代码后自动重载呢？

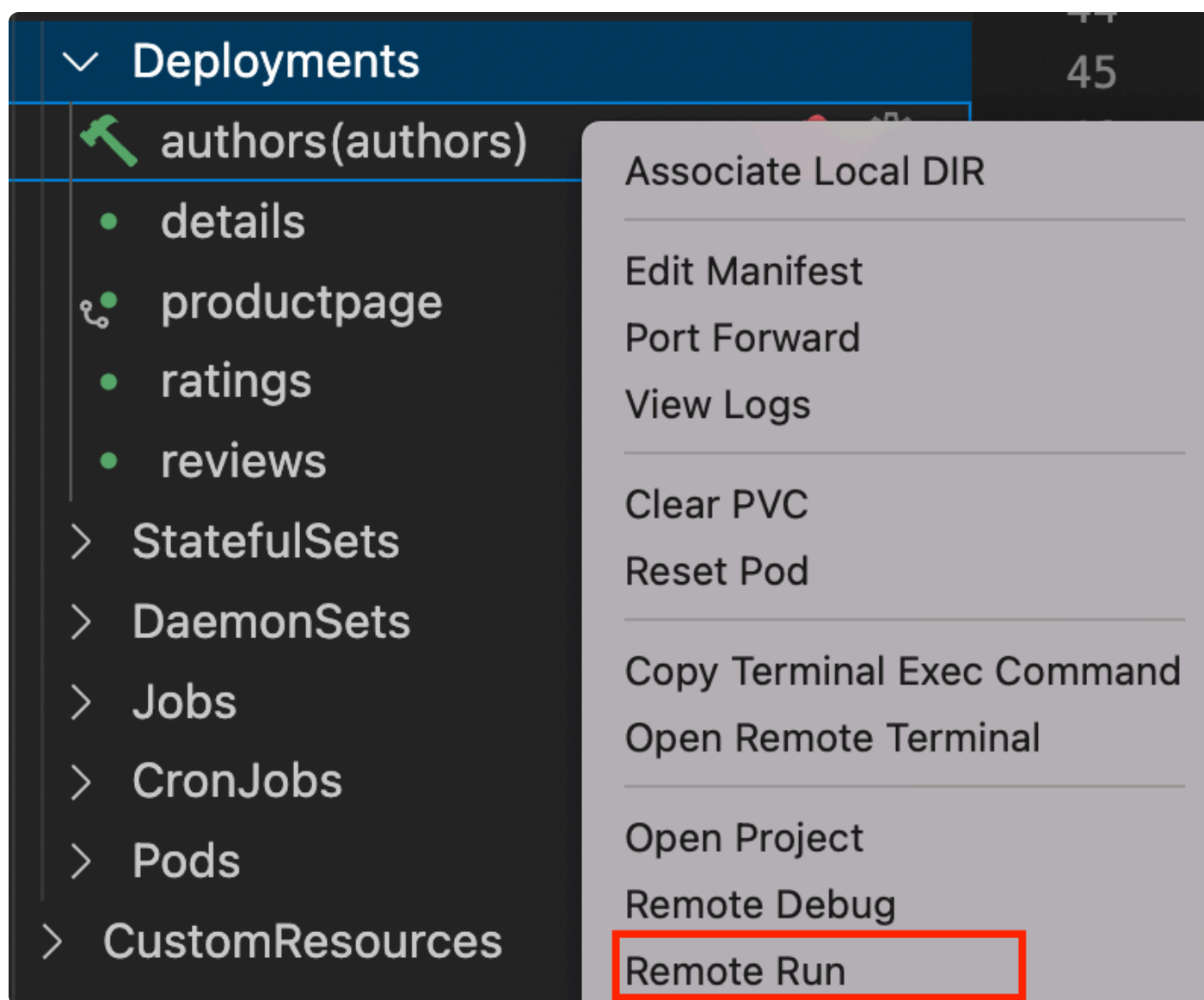
Nocalhost 同样也为我们提供了和语言无关的容器热加载，也就是说，当本地有任何代码变更时，Nocalhost 都会自动帮助我们重启容器内的业务进程，达到容器热加载的目的。

接下来，我们一起来体验这个功能。

首先，在当前 VS Code 窗口中重新打开 Nocalhost 插件，找到 authors 服务。此时，你将看到该服务左侧有一个“绿色锤子”图标，这表示这个服务正在开发模式当中，如下图所示。



接下来，右击 authors 服务，选择最后一个选项 Remote Run。

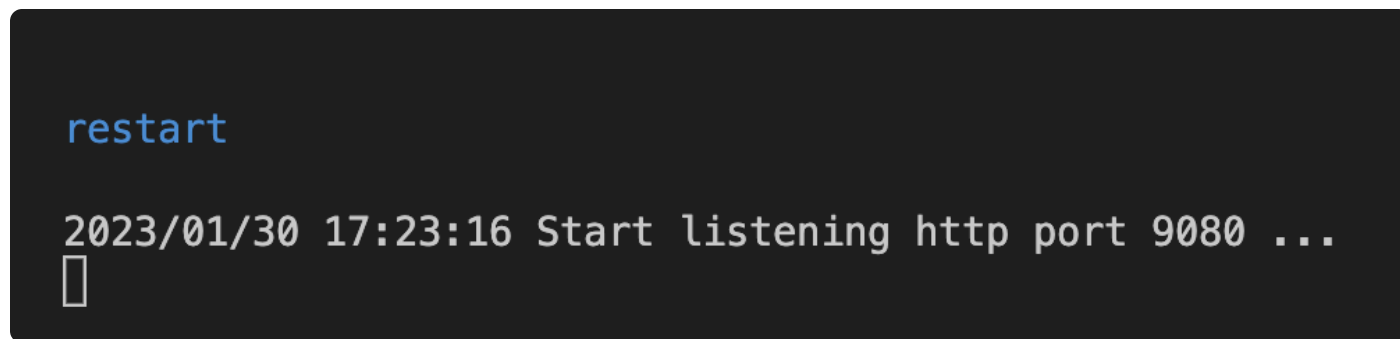


注意，在点击 Remote Run 之前，一定要先确保已经通过 Ctrl+C 的方式手动停止了容器内的业务进程，这可以避免重复运行业务进程导致的端口冲突。

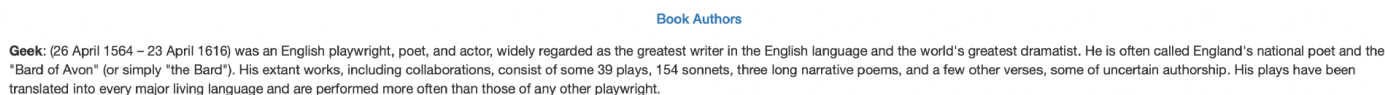
现在，Nocalhost 将自动开启一个新的终端，并自动启动业务进程，你也可以通过右侧来切换不同的终端。



接下来，尝试继续修改 app.go 的 54 行，例如删除“bang”字符串并**保存**。此时，你将看到 Nocalhost 发现了修改并自动重启了容器内的业务进程，如下图所示。



然后，重新打开浏览器并刷新页面，你会发现新的修改已经实时生效了。



有一些同学可能会有疑问，Nocalhost 怎么知道我的业务的启动命令呢？答案是通过为 Nocalhost 配置启动命令。你可以通过点击 authors 服务右侧的“设置”按钮，在弹出的对话框中选择“取消”来查看配置文件中的 command.run 字段。实际上，Nocalhost 是通过运行配置的 run.sh 脚本来启动业务的。

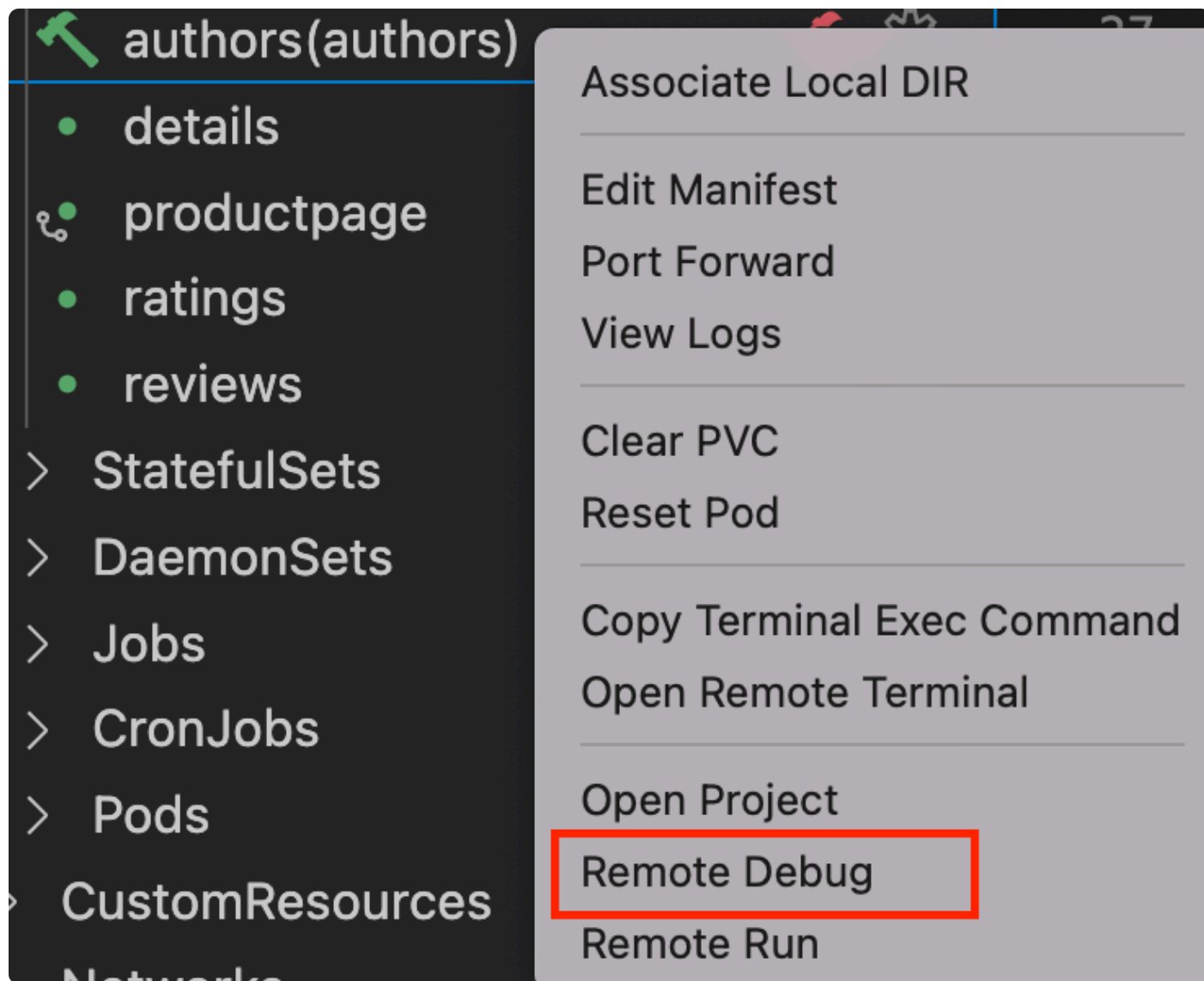
最后，你可以在终端窗口中通过 Ctrl+C 的方式来中断容器热加载。

到这里，Nocalhost 容器热加载的全过程就已经体验完了。

一键调试

除了容器热加载以外，Nocalhost 还为我们提供了便利的一键远程调试功能。

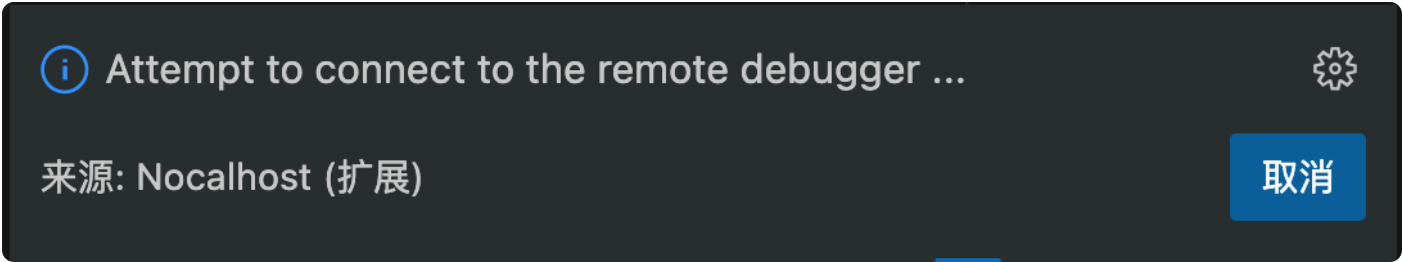
同样地，找到 authors 服务，右击选择“Remote Debug”来进入远程调试。



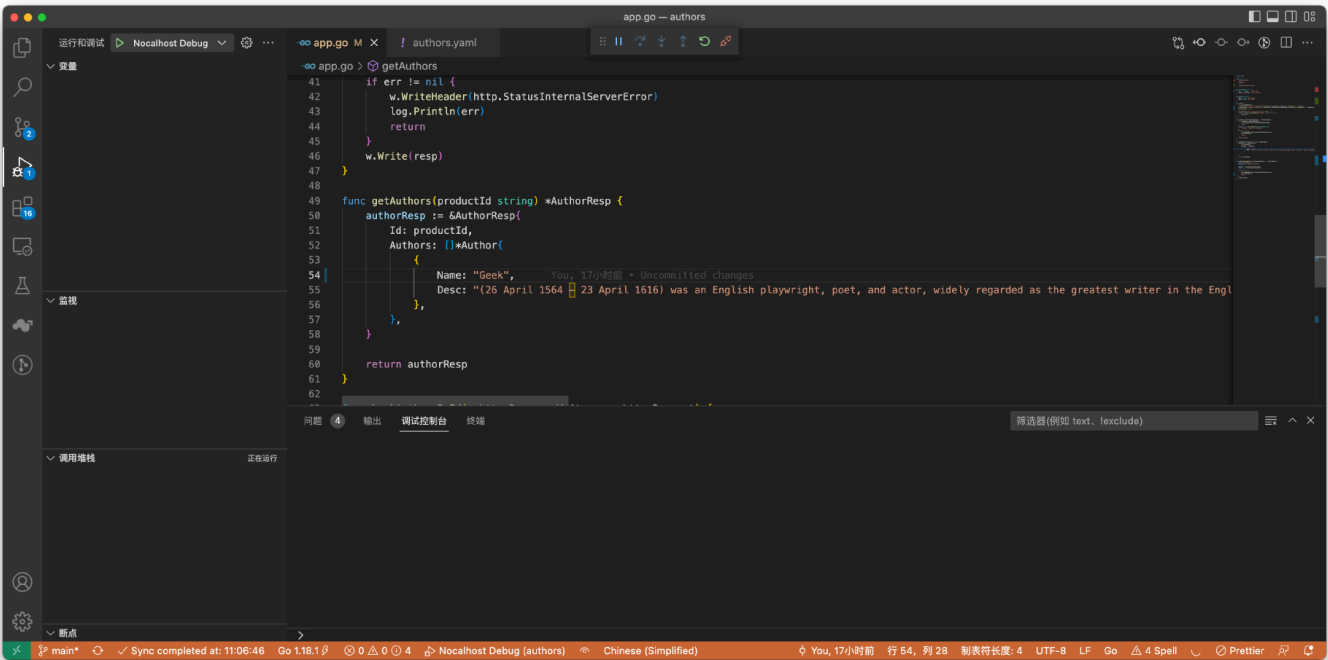
接下来，Nocalhost 就会以调试模式启动业务进程，然后通过 Kubernetes 端口转发的方式将远端的调试端口转发到本地，并控制调试器连接到调试端口。

需要注意的是，由于 authors 服务是 Golang 编写的，所以调试依赖于本地的 Golang 开发工具，如果你的电脑里没有 Golang 开发环境，Nocalhost 将提示你安装相关工具和插件。

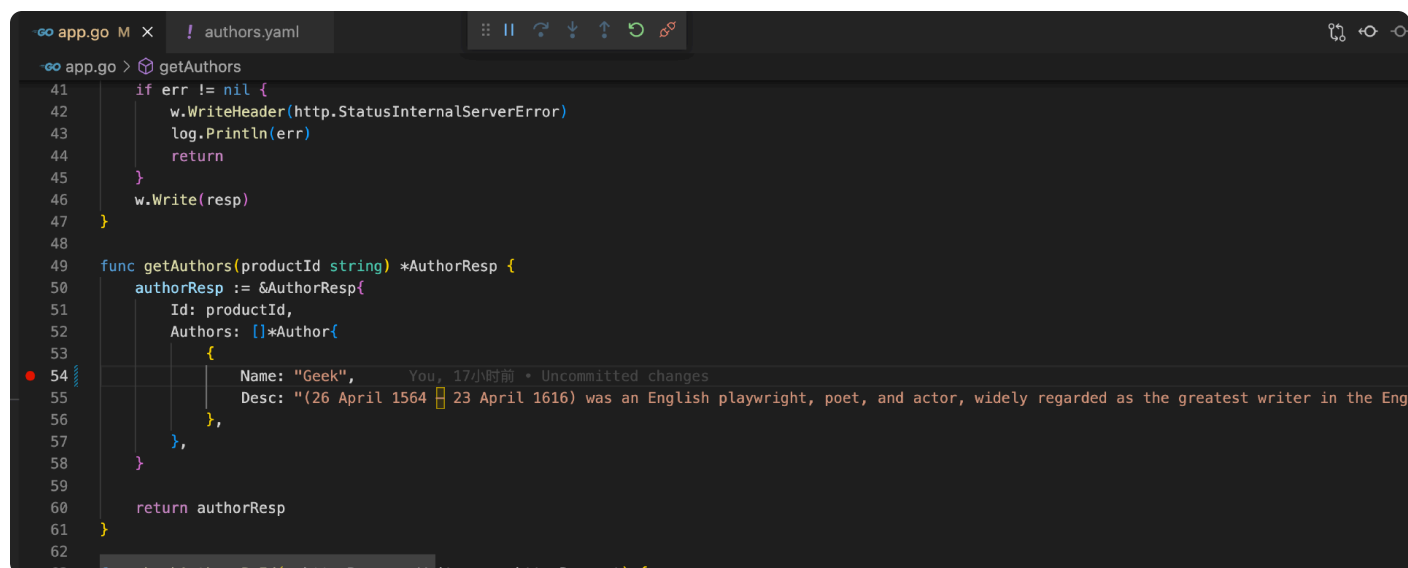
进入调试后，你将看到 VS Code 窗口右下角出现准备连接调试器，如下图所示。



大约等待十几秒钟后，如果弹出了 VS Code 的调试窗口，说明已经成功连接了远端的调试进程，调试界面如下图所示。

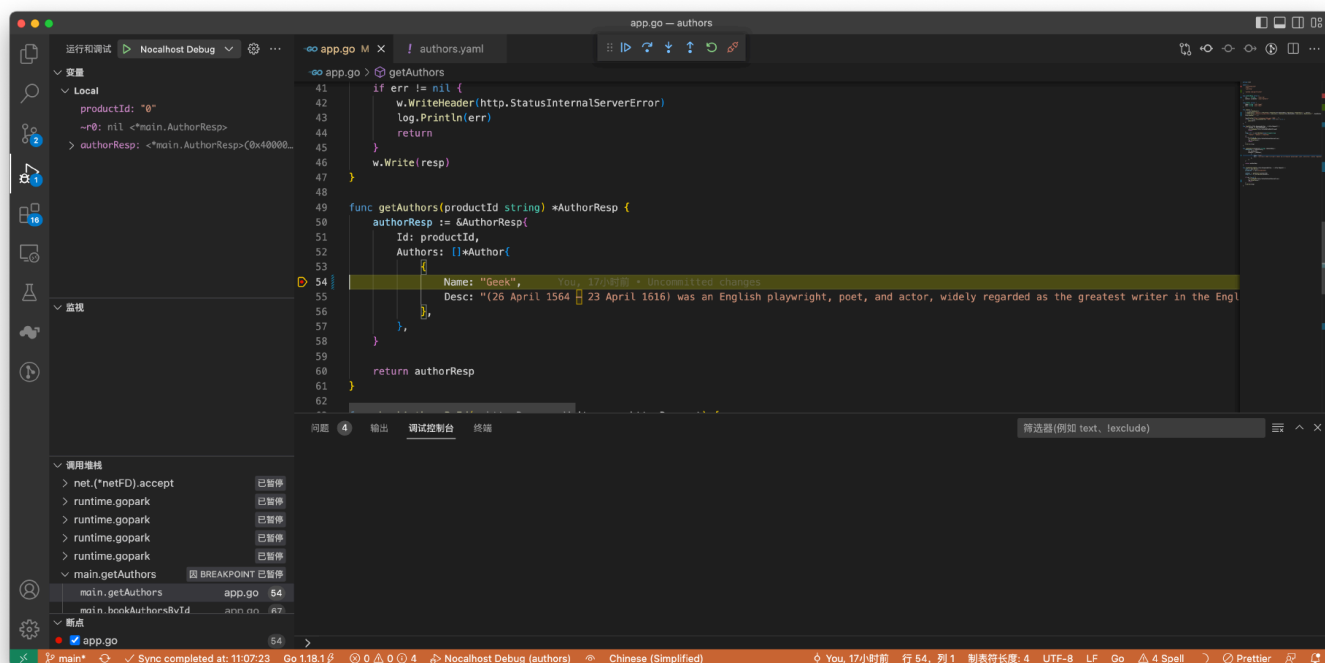


接下来，我们可以打开 app.go 文件，并在第 54 行右侧打一个断点，此时，在行数的右侧将出现一个红色的圈圈，代表我们打断点的位置。



```
app.go M X ! authors.yaml
app.go > getAuthors
41 if err != nil {
42     w.WriteHeader(http.StatusInternalServerError)
43     log.Println(err)
44     return
45 }
46 w.Write(resp)
47 }
48
49 func getAuthors(productId string) *AuthorResp {
50     authorResp := &AuthorResp{
51         Id: productId,
52         Authors: []*Author{
53             {
54                 Name: "Geek",
55                 Desc: "(26 April 1564 23 April 1616) was an English playwright, poet, and actor, widely regarded as the greatest writer in the Eng
56             },
57         },
58     }
59
60     return authorResp
61 }
62
```

现在，你可以**重新返回浏览器并刷新页面**，这时候，VS Code 调试器将停留在我们打断点的位置，并且在左侧调试菜单栏中展示相关的变量信息。



通过上面的操作，我们便完成了对 authors 服务的一键调试。

在这个调试例子中，如果你用的是 M1 芯片的 Mac，那么你可能会发现在调试过程中 VS Code 的调试器一直无法连接到远端容器，这时候，你还需要进行下面的操作。

在 Nocalhost 插件中点击 authors 服务的“设置”按钮进入服务的开发配置页，并将 image 字段修改为 okteto/golang:1.19，然后，点击“红色锤子”退出 authors 服务的开发模式，退出完成后，再点击“Remote Debug”来进入调试模式即可。

最后，要退出调试模式，你可以切换到 VS Code 终端菜单，并通过 Ctrl+C 的方式来终止调试进程。

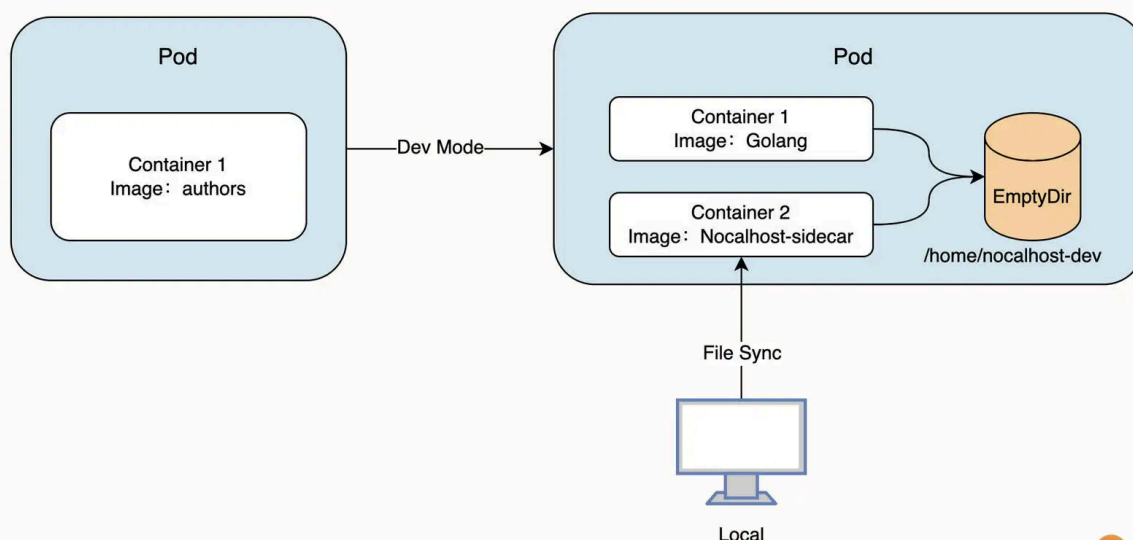
原理解析

了解 Nocalhost 的基本使用后，我们来简单看一下它的基本原理。

你可以先思考这样一个问题：在进入开发模式后，我们为什么能以源码的方式在容器里启动业务进程呢？

这个问题看似简单，但却是 Nocalhost 远程开发的核心原理。实际上，我们之所以能在容器内以源码的方式启动业务进程，是因为容器的配置、Secret、服务依赖等并没有变，我们只是将二进制方式启动的业务进程替换成了以源码的方式启动，这是实现远程开发的基础。

在进入开发模式后，Nocalhost 会将容器的镜像替换为开发镜像，并增加用来进行文件同步的 Sidecar 容器，为了更好地帮助你理解，我结合 Kubernetes 画了一张架构图放在了文稿中，你可以看一下。



类似 authors 服务这种编译型语言编写的业务应用，在构建镜像的时候并不会将源码打包到镜像内，并且一般不包含特定的语言开发和编译工具。所以，为了解决这两个问题，在进入开发模式后，Nocalhost 将原来的业务镜像替换为了包含开发和编译工具的 Golang 镜像，并额外增加了 Sidecar 容器，用来将本地的源码同步到容器内。

此外，开发容器和 Sidecar 容器共享同一个卷存储，这样，当本地代码同步到 Sidecar 容器后，开发容器同样也能够访问到源码，这么做的好处是能够将开发镜像和文件同步功能解耦，你可以随意更换任何开发镜像。

那么，本地又是如何连接到 Sidecar 容器以及进行文件同步的呢？答案是端口转发。

Sidecar 容器在启动后，将启动 Syncthing 文件同步服务，并监听在容器特定端口。随后，Nocalhost 将容器的文件同步服务通过端口转发的方式和本地打通，并同时在本地启动了 Syncthing 客户端，实现了文件的单向和双向同步的功能。

以上就是 Nocalhost 实现远程开发的基本原理了。

总结

总结一下，这节课，我向你介绍了 Nocalhost 的基本使用方法，包括容器热加载和一键调试。使用 Nocalhost 开发 Kubernetes 应用，可以大大地减少开发循环反馈所需要等待的时间，提升开发效率。

在实战环节中，我通过部署示例应用来向你介绍了使用 Nocalhost 的操作细节，当然，由于示例应用内置了开发配置，所以我们整个体验过程是相对顺畅的。当你需要使用 Nocalhost 来开发真实的业务应用时，你可以在 Nocalhost 插件中找到对应的工作负载，并点击“设置”按钮为它配置基本的开发参数就可以进入开发模式了，在配置的过程中，你还可以借助 Nocalhost 提供的网页配置工具来降低开发参数的配置门槛。

此外，我还为你简单介绍了 Nocalhost 的基本原理，它主要是通过替换容器镜像为开发镜像，并增加文件同步的 Sidecar 容器来提供开发工具链和源码。这样，你只需要为待开发的服务配置合适的开发镜像而不需要关注文件同步。

最后，除了远程开发模式，Nocalhost 还提供了 Proxy 代理模式、Service Mesh 和 Duplicate 复制开发模式，不过使用场景相对较少，感兴趣的同学可以在 [🔗 这个文档](#) 查看相关资料。

思考题

最后，给你留一道思考题吧。

除了 Nocalhost 这种开发方式以外，在 Kubernetes 环境下你还了解或实践过其他的开发方式吗？对比 Nocalhost 的开发方式，你认为它们有哪些优劣呢？

欢迎你给我留言交流讨论，你也可以把这节课分享给更多的朋友。我们下节课见。

AI智能总结

Nocalhost是一个能够帮助开发者在Kubernetes环境下实现秒级开发体验的工具。本文介绍了Nocalhost的基本使用方法，包括容器热加载和一键调试功能。通过Nocalhost，开发者可以在容器内实时修改代码并立即查看效果，实现了远程开发的实时性和便捷性。文章还解析了Nocalhost的基本原理，包括容器镜像替换和文件同步的实现方式。此外，Nocalhost还提供了Proxy代理模式、Service Mesh和Duplicate复制开发模式。最后，读者被引导思考除了Nocalhost之外，在Kubernetes环境下还有哪些开发方式，以及它们与Nocalhost的优劣对比。整体而言，本文通过介绍Nocalhost的使用方法和优势，为读者提供了一种加速开发循环反馈的解决方案。

全部留言 (8)

最新 精选



(͡° ͜ʖ ͡°)

2023-02-27 来自上海

按网上和老师说的修改时间有几个问题

- 1 依赖镜像，不是所有的镜像都支持TZ="Asia/Shanghai"，我们不想业务被镜像限制住
- 2 时区修改能精确到分钟吗，我们的定时器不一定是整点运行，可能是00:00:01、04:01:00这些分散的时间点，避免定时任务同一时刻集中触发
- 3 类似用户上线后给一个累计在线时长的小奖励，这种逻辑需要把时间向后调，这是跟着配置来的，根时区没关系。比如我们有5分钟、10分钟、半小时、1小时，以前qa重启4次改4次时间 5分钟就把所有情况都测完了。按老师说的改时区、我想不出来怎么弄。。。
- 2 qa肯定是不会用k8s的，让他们修改窗口的启动参数或者重新做个镜像也不现实，以前停服、改时间、重启没什么操作难度，1分钟就测好了。改了k8s，以前1分钟的工作要重新打包、上传镜像、重新运行、怎么看都要以10分钟为单位起算。这个成本公司不会接受。有没有即时生效、简单便捷的方法呢？

作者回复: 总体的思考原则是，QA 测试的困难不能影响业务架构的升级和迁移，迁移的好处远远大于给测试带来的影响。

这种测试场景本身就比较复杂，我比较建议一并将这个测试流程改造成自动化测试进行，把复杂度内聚到测试程序里面。另一个解决思路是把定时器解耦到外部服务，而不依赖于容器的时间。

至于修改时间的问题，确实会有你提到的种种困难，不过也并不是没有解决方案，你可以看这个项目：<https://github.com/wolfcw/libfaketime>。它可以实现在单个容器里伪造任何你想要的时间，不过操作起来比较复杂，所以最好让 QA 把这部分测试逻辑改造成自动化。



👍 2



(͡° ͜ʖ ͡°)

2023-02-27 来自上海

请老师解决一个阻碍我们尝试kubernetes的疑惑
docker和k8s里怎么修改时间？

我们公司比较传统，开发好的功能由qa的同事测试，qa同事在自己的机器上部署应用，应用中的定时或延迟任务都是停服、修改本地时间然后重启测试。

比如有个凌晨12点的任务，qa停了自己的服，改下自己机器的时间到23:59:00，然后重启服务器，等一分钟到12点了看任务有没有成功执行。

网上搜索到的修改docker时间的方法就两个，一个是把系统时间相关的类库挂载到容器里，另一个是修改时区。这两个方法对我们的测试并不可用。极客时间的其他有关docker、kubernetes的专栏我也有订阅，但老师们都没有提到过这个问题，我很好奇难道大家都不用定时器的吗？

另外这节课里提到的Nocalhost也是同一个疑惑，我修改了一段定时器的触发逻辑，怎么在pod里修改时间然后测试是否正常触发了呢？

课程听到现在一直没有找到合适的地方提问，看课表这个专栏也快结束了，希望老师在结课前能指点一下我，让我们公司能在云原生开发从0到1

作者回复: 你可以简单的把 Pod 理解为虚拟机就可以了，虚拟机怎么改时区和时间，在 Pod 里就怎么改。

比如，如果你的基础镜像是 Debian 或 CentOS，那么可以通过设置 TZ 环境变量来改时区，例如：TZ=Asia/Shanghai。

其次，还可以在构建容器镜像的时候通过修改 Dockerfile 也可以修改时区和时间，参考这个链接：<https://dev.to/0xbf/set-timezone-in-your-docker-image-d22>。



👍 2



黑鹰

2023-03-03 来自北京

nocalhost-docker.pkg.coding.net/nocalhost/dev-images/golang 这个镜像拉取特别慢，国内网络和VPN 经常拉取失败。

```
Warning Failed    47m (x4 over 80m) kubelet          Failed to pull image "nocalhost
-docker.pkg.coding.net/nocalhost/dev-images/golang:latest": rpc error: code = Unkno
wn desc = context deadline exceeded
```

```
Warning Failed    47m (x4 over 80m) kubelet
```

```
Error: ErrImagePull
```

```
Warning Failed    46m (x5 over 76m) kubelet
```

```
Error: ImagePullBackOff
```

Normal Pulling 30m (x5 over 82m) kubelet Pulling image "nocalhost-docker.pkg.coding.net/nocalhost/dev-images/golang:latest"
Normal BackOff 25m (x12 over 76m) kubelet Back-off pulling image "nocalhost-docker.pkg.coding.net/nocalhost/dev-images/golang:latest"

作者回复: 可以换成 dockerhub 的 golang 镜像, 只要有编译环境即可。



👍 1



林龍

2023-02-27 来自广东

请问一下, 本篇文章中考虑到第28章的一个安全性的问题。28中我觉得一个观点让我受益匪浅就是持续集成工具不应该可以直接操作k8s, 这也是gitops我觉得的核心, 本篇文章是通过本地连接, 一是考虑到k8s不应该外网访问, 第二是不应该所有人(开发)都能操作k8s。

作者回复: 生产环境是需要杜绝直连 K8s 集群的。开发环境为了提高开发效率可以考虑开放, 并通过 IP 白名单的方式兼顾安全性。



👍 1



bingo

2023-03-02 来自吉林

老师, 使用nocalhost进行本地调试不知道什么原因内存飙升, idea关了内存还是降不下去, 请问我该从哪些方面找找问题

作者回复: 可能是 Nocalhost 本身的问题, IDEA 插件占用内存过高, VSCODE 插件性能会更好。



👍



(ε)

2023-02-27 来自上海

还有一个问题

网上搜到的一些说法, 我在一个容器里修改时间, 会影响到外面宿主主机。

如果我们用namespace把不同团队做隔离, 但大家可能会被分配在一台node上吧, 这样的话, 一个人修改了自己的服务时间, 会影响到其他人。自己的服务器跑着跑着莫名其妙的时间跳了, 出来一堆bug。

作者回复: 已回复~

共 3 条评论 >



Geek_45a572

2024-04-18 来自陕西

Kubeconfig分发是一个问题，多个团队共享一个集群



Demon.Lee

2023-12-14 来自安徽

老师你好，自从 2023.4.24之后，我看 nocalhost 在 github 上就没有发布新版本了，是没人维护了吗，还是有其他原因。

